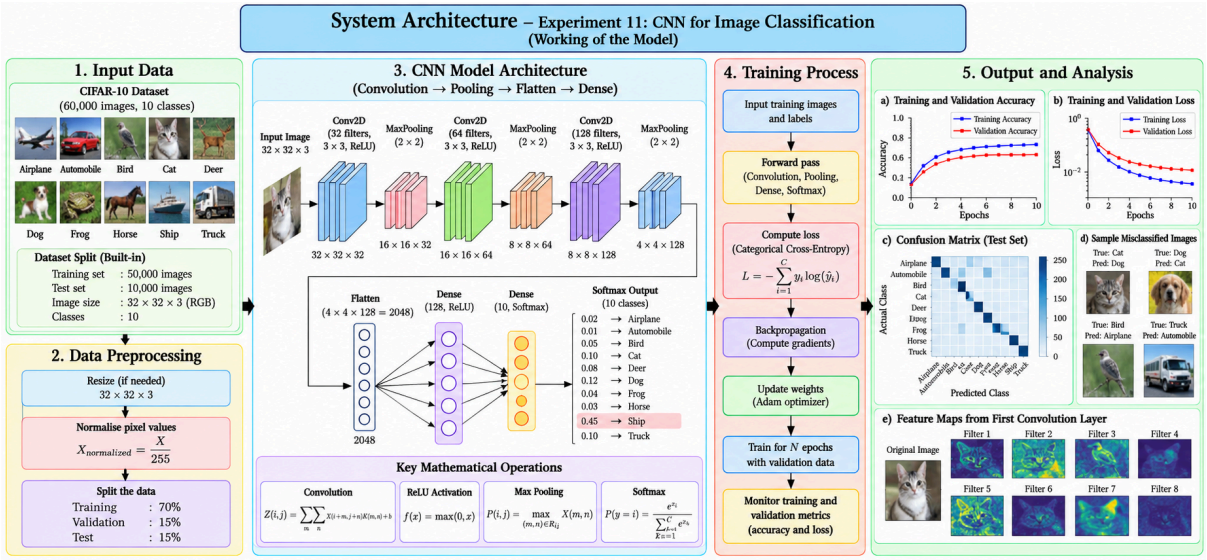
Experiment 11: CNN for Image Classification

1. Aim

To design and implement a Convolutional Neural Network (CNN) using TensorFlow and Keras for multi-class image classification. To load, resize, and normalize images, divide the dataset into training, validation, and testing sets, train the CNN using categorical cross-entropy loss, evaluate its performance using accuracy and a confusion matrix, visualize misclassified images, and examine feature maps produced by the first convolutional layer using Matplotlib.



2. Mathematical Formulas

1. Image Normalization

$$\boxed{X_{\text{normalized}} = \frac{X}{255}}$$

2. Convolution Operation

$$\boxed{Z(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b}$$

Where X is the input image, K is the convolution filter, and b is the bias.

3. ReLU Activation Function

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

4. Max Pooling

$$\boxed{P(i,j) = \max_{(m,n) \in R_{ij}} X(m,n)}$$

5. Softmax Activation Function

$$\boxed{P(y=i) = \frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}}$$

6. Categorical Cross-Entropy Loss

$$\boxed{L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)}$$

7. Accuracy

$$\boxed{\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100}$$

CODE

```
# Experiment 11: CNN for Image Classification

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix, classification_report,
    ConfusionMatrixDisplay, accuracy_score
)

np.random.seed(42)
tf.random.set_seed(42)

# ==========================================
# 1. LOAD CIFAR-10 DATASET
# ==========================================

(X_all, y_all), (X_test, y_test) = (
    tf.keras.datasets.cifar10.load_data()
)

y_all = y_all.ravel()
y_test = y_test.ravel()

class_names = [
    "Airplane", "Automobile", "Bird", "Cat", "Deer",
    "Dog", "Frog", "Horse", "Ship", "Truck"
]

print("Training dataset shape:", X_all.shape)
print("Test dataset shape:", X_test.shape)
print("Number of classes:", len(class_names))

# ==========================================
# 2. NORMALIZE AND SPLIT THE DATA
# ==========================================

# Convert pixel values from [0,255] to [0,1]
X_all = X_all.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

X_train, X_val, y_train, y_val = train_test_split(
    X_all, y_all,
    test_size=0.15,
    random_state=42,
    stratify=y_all
)

print("\nTraining images:", len(X_train))
print("Validation images:", len(X_val))
print("Test images:", len(X_test))
```

```
# ========================================
# 3. DISPLAY SAMPLE IMAGES
# ========================================

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis("off")

plt.suptitle("Sample CIFAR-10 Images")
plt.tight_layout()
plt.show()

# ========================================
# 4. BUILD THE CNN MODEL
# ========================================

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),

    tf.keras.layers.Conv2D(
        32, (3, 3), activation="relu",
        padding="same", name="conv1"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(
        64, (3, 3), activation="relu",
        padding="same"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Conv2D(
        128, (3, 3), activation="relu",
        padding="same"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# ========================================
# 5. TRAIN THE CNN
# ========================================
```

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=10,
    batch_size=64,
    verbose=1
)

# ========================================
# 6. PLOT ACCURACY AND LOSS CURVES
# ========================================

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy")
plt.legend()
plt.grid(alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss")
plt.legend()
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

# ========================================
# 7. EVALUATE ON TEST DATA
# ========================================

test_loss, test_accuracy = model.evaluate(
    X_test, y_test, verbose=0
)

probabilities = model.predict(X_test, verbose=0)
y_pred = np.argmax(probabilities, axis=1)

print("\nTest Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    labels=np.arange(10),
    target_names=class_names,
    zero_division=0
))
```

```
# ========================================
# 8. PLOT CONFUSION MATRIX
# ========================================

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)
display.plot(cmap="Blues", xticks_rotation=45, values_format="d")
plt.title("CNN Confusion Matrix")
plt.tight_layout()
plt.show()

# ========================================
# 9. DISPLAY MISCLASSIFIED IMAGES
# ========================================

wrong_indices = np.where(y_pred != y_test)[0]

plt.figure(figsize=(12, 6))

for position, idx in enumerate(wrong_indices[:10]):
    plt.subplot(2, 5, position + 1)
    plt.imshow(X_test[idx])
    plt.title(
        f"True: {class_names[y_test[idx]]}\n"
        f"Pred: {class_names[y_pred[idx]]}"
    )
    plt.axis("off")

plt.suptitle("Sample Misclassified Images")
plt.tight_layout()
plt.show()

print("Total misclassified images:", len(wrong_indices))

# ========================================
# 10. VISUALIZE FIRST CONVOLUTION FEATURE MAPS
# ========================================

feature_model = tf.keras.Model(
    inputs=model.inputs,
    outputs=model.get_layer("conv1").output
)

sample_image = X_test[0:1]
feature_maps = feature_model.predict(sample_image, verbose=0)

plt.figure(figsize=(12, 8))

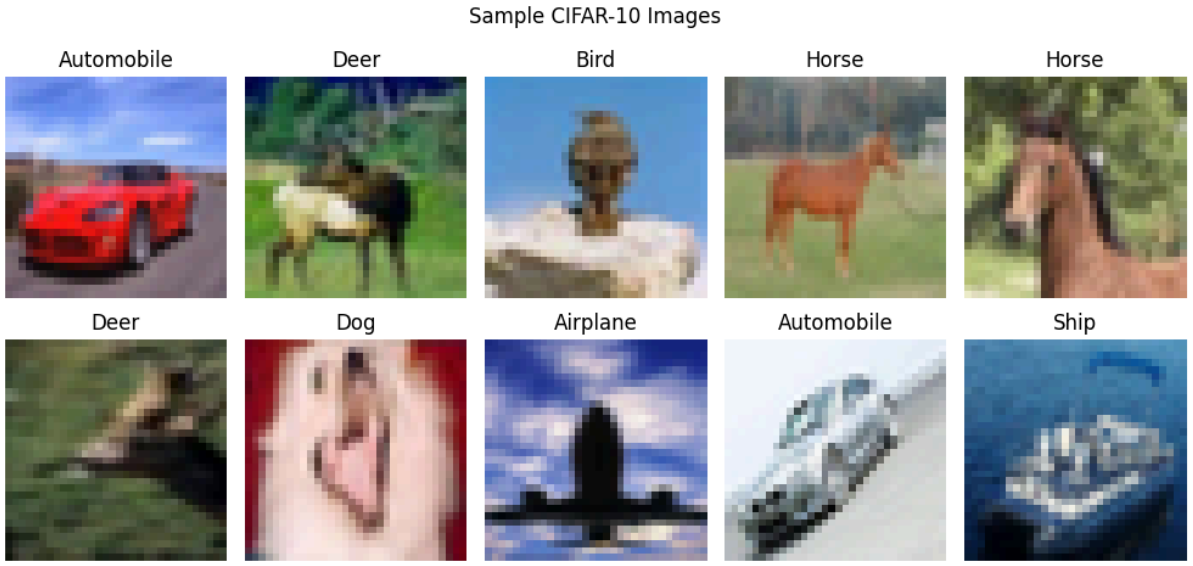
for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(feature_maps[0, :, :, i], cmap="viridis")
    plt.title(f"Filter {i + 1}")
    plt.axis("off")
```

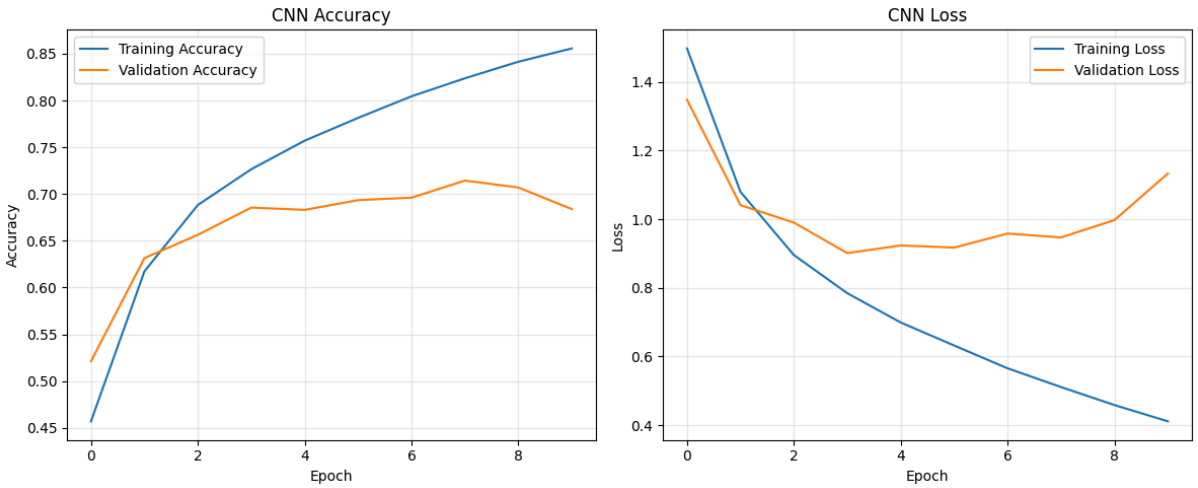
```
plt.suptitle("Feature Maps from First Convolution Layer")
plt.tight_layout()
plt.show()
# Display the original image used for feature extraction
plt.figure(figsize=(3, 3))
plt.imshow(X_test[0])
plt.title(f"Original: {class_names[y_test[0]]}")
plt.axis("off")
plt.show()
```

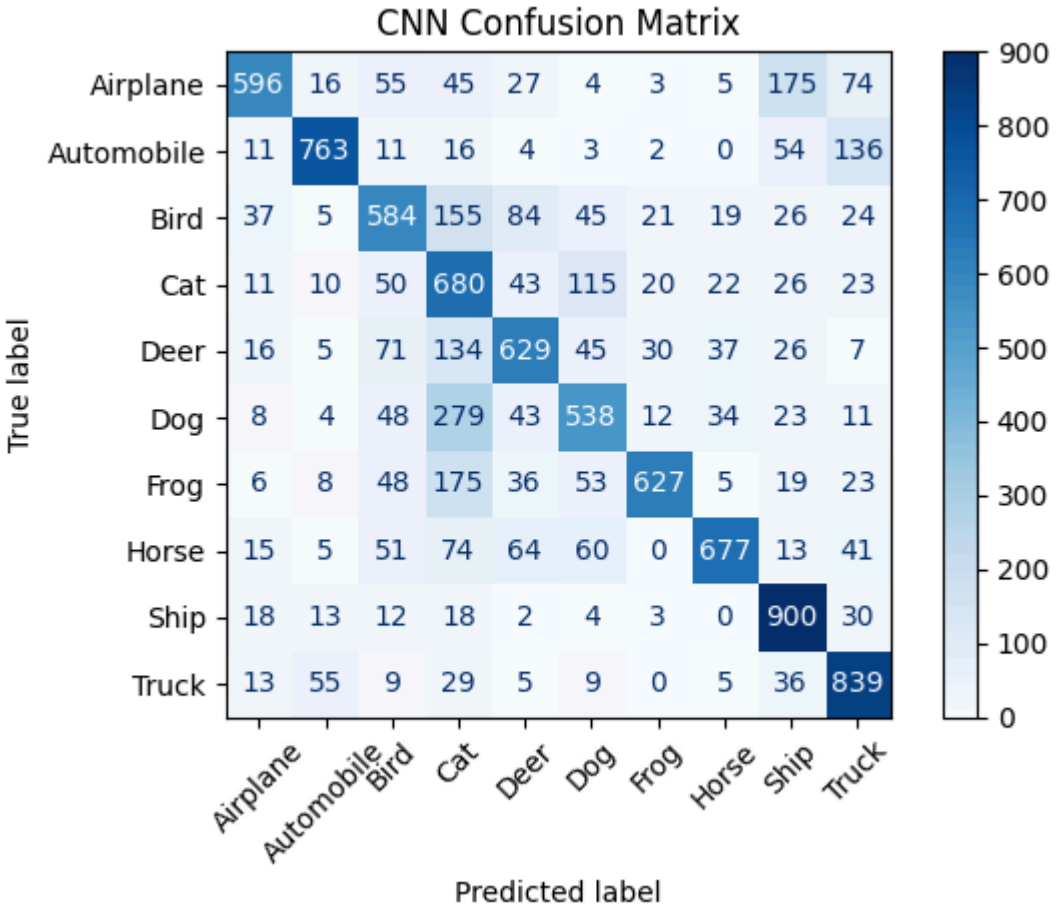
OUTPUT

Training dataset shape: (50000, 32, 32, 3)
Test dataset shape: (10000, 32, 32, 3)
Number of classes: 10

Training images: 42500
Validation images: 7500
Test images: 10000



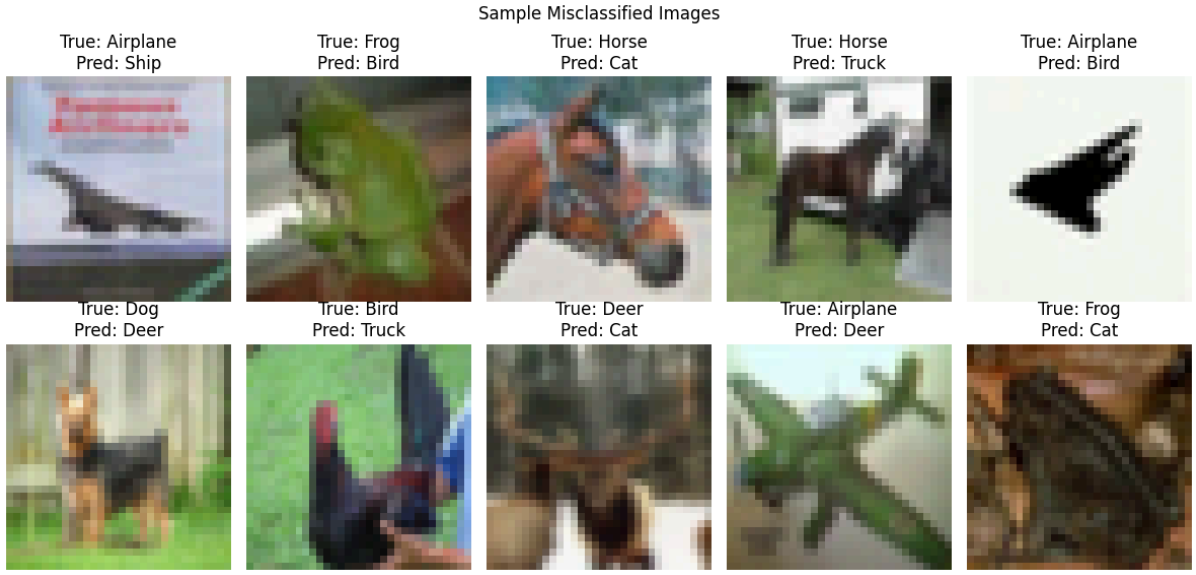

CNN Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
CNN Loss
Training Loss
Validation Loss
Loss
Epoch

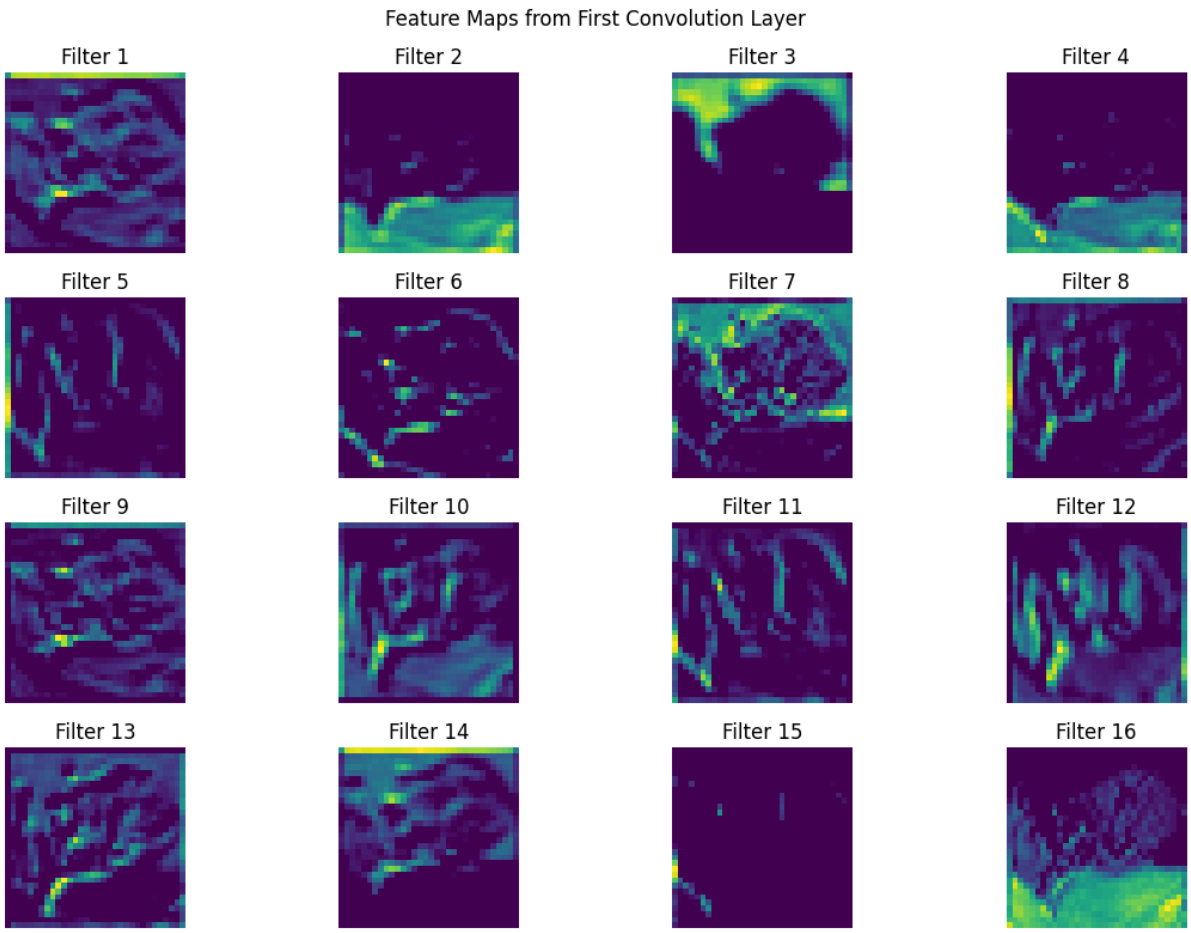

CNN Confusion Matrix
Airplane 596 16 55 45 27 4 3 5 175 74
Automobile 11 763 11 16 4 3 2 0 54 136
Bird 37 5 584 155 84 45 21 19 26 24
Cat 11 10 50 680 43 115 20 22 26 23
Deer 16 5 71 134 629 45 30 37 26 7
Dog 8 4 48 279 43 538 12 34 23 11
Frog 6 8 48 175 36 53 627 5 19 23
Horse 15 5 51 74 64 60 0 677 13 41
Ship 18 13 12 18 2 4 3 0 900 30
Truck 13 55 9 29 5 9 0 5 36 839
Airplane Automobile Bird Cat Deer Dog Frog Horse Ship Truck
True label
Predicted label
900 800 700 600 500 400 300 200 100 0

Test Loss: 1.1354
Test Accuracy: 68.33 %

Classification Report:

	precision	recall	f1-score	support
Airplane	0.82	0.60	0.69	1000
Automobile	0.86	0.76	0.81	1000
Bird	0.62	0.58	0.60	1000
Cat	0.42	0.68	0.52	1000
Deer	0.67	0.63	0.65	1000
Dog	0.61	0.54	0.57	1000
Frog	0.87	0.63	0.73	1000
Horse	0.84	0.68	0.75	1000
Ship	0.69	0.90	0.78	1000
Truck	0.69	0.84	0.76	1000
accuracy			0.68	10000
macro avg	0.71	0.68	0.69	10000
weighted avg	0.71	0.68	0.69	10000





5. RESULT

A Convolutional Neural Network was implemented using TensorFlow and Keras for multi-class image classification on the CIFAR-10 dataset. Images were normalized and divided into training, validation, and test sets. The CNN was trained using convolutional layers, max pooling, dense layers, the Adam optimizer, and categorical classification loss. Training and validation curves, a confusion matrix, sample misclassifications, and first-layer feature maps were visualized. The model's classification performance was evaluated using test accuracy, precision, recall, and F1-score.